

Laporan Proyek OpenGL
Visualisasi Model Tung Tung Sahur dengan Phong Shading

Anggota Kelompok

No.	Nama	NIM
1	Nama Anggota 1	NIM Anggota 1
2	Nama Anggota 2	NIM Anggota 2
3	Nama Anggota 3	NIM Anggota 3

1 Objektif Tugas

Objektif dari tugas ini adalah membuat program visualisasi 3D menggunakan OpenGL yang memenuhi tiga bagian utama dari spesifikasi tugas, yaitu:

1. Mengimplementasikan vertex shader dan fragment shader dengan model pencahayaan Phong.
2. Membuat fungsi untuk memuat model 3D ke dalam scene, lalu menggunakan VBO dan VAO agar data model dapat diproses oleh GPU.
3. Membuat pengaturan matriks model, view, dan projection secara terpisah.

Rencana implementasi dilakukan secara bertahap. Pertama, program menyiapkan window OpenGL, memuat file model `TungTungTungSahur.obj`, dan menyimpan data posisi serta normal vertex. Kedua, data tersebut dikirim ke GPU memakai VAO dan VBO. Ketiga, dibuat shader untuk menghitung warna objek menggunakan komponen ambient, diffuse, dan specular dari model Phong. Terakhir, program memisahkan perhitungan transformasi ke dalam fungsi model matrix, view matrix, dan projection matrix agar alur rendering lebih jelas.

2 Setup

Program ditulis menggunakan bahasa C++ dan dibangun melalui `Makefile`. Library yang digunakan adalah:

- **OpenGL**: API utama untuk rendering grafis 3D.
- **GLFW**: digunakan untuk membuat window, membuat OpenGL context, dan membaca input keyboard.

- **GLEW**: digunakan untuk memuat fungsi-fungsi OpenGL modern setelah context dibuat.
- **GL**: library OpenGL sistem yang ditautkan saat proses build.

Konfigurasi build berada di file **Makefile**. Program dikompilasi dengan standar C++17 dan dependency GLFW/GLEW diambil dari **pkg-config**.

```
CXXFLAGS := -std=c++17 -Wall -Wextra -O2
CPPFLAGS := $(shell pkg-config --cflags glfw3 glew)
LDLIBS := $(shell pkg-config --libs glfw3 glew) -lGL -lm
```

Program dapat dibangun dan dijalankan dengan perintah:

```
make
make run
```

3 Formula Matematika yang Digunakan

Bagian ini menjelaskan rumus matematika yang langsung berkaitan dengan implementasi program. Rumus-rumus ini digunakan pada perhitungan vektor, pemuatan normal model, kamera, transformasi matriks, dan pencahayaan Phong.

3.1 Operasi Vektor

Program menggunakan dot product untuk menghitung kedekatan arah dua vektor, misalnya antara normal permukaan dan arah cahaya.

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z \quad (1)$$

Cross product digunakan untuk mendapatkan normal face ketika file OBJ tidak menyediakan normal. Jika suatu segitiga memiliki titik $\mathbf{p}_0$, $\mathbf{p}_1$, dan $\mathbf{p}_2$, maka normal face dihitung dengan:

$$\mathbf{n}_{face} = \frac{(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)}{\|(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)\|} \quad (2)$$

Normalisasi vektor dilakukan agar panjang vektor menjadi satu:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{\mathbf{v}}{\sqrt{v_x^2 + v_y^2 + v_z^2}} \quad (3)$$

3.2 Bounding Box, Center, dan Scale Model

Loader OBJ menghitung titik minimum dan maksimum dari semua vertex untuk membuat bounding box model. Titik tengah model dihitung dengan:

$$\mathbf{c} = \frac{\mathbf{p}_{min} + \mathbf{p}_{max}}{2} \quad (4)$$

Ukuran terbesar model dihitung dari extent:

$$\mathbf{e} = \mathbf{p}_{max} - \mathbf{p}_{min} \quad (5)$$

$$e_{max} = \max(e_x, e_y, e_z) \quad (6)$$

Skala model pada program dibuat agar objek memiliki ukuran visual yang sesuai di scene:

$$s = \begin{cases} \frac{2.8}{e_{max}}, & e_{max} > 0 \\ 1, & e_{max} = 0 \end{cases} \quad (7)$$

3.3 Model Matrix

Model matrix digunakan untuk memindahkan model ke pusat scene dan mengubah ukurannya. Implementasi program menggunakan translasi ke arah negatif center model, kemudian scaling:

$$M = S(s) \cdot T(-\mathbf{c}) \quad (8)$$

Posisi vertex dalam koordinat dunia dihitung dengan:

$$\mathbf{p}_{world} = M \cdot \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} \quad (9)$$

3.4 View Matrix

View matrix dibuat dengan metode `lookAt`. Jika posisi kamera adalah **eye**, target adalah **center**, dan arah atas adalah **up**, maka basis kamera dihitung sebagai:

$$\mathbf{f} = \frac{\mathbf{center} - \mathbf{eye}}{\|\mathbf{center} - \mathbf{eye}\|} \quad (10)$$

$$\mathbf{s} = \frac{\mathbf{f} \times \mathbf{up}}{\|\mathbf{f} \times \mathbf{up}\|} \quad (11)$$

$$\mathbf{u} = \mathbf{s} \times \mathbf{f} \quad (12)$$

Berdasarkan basis tersebut, matriks view menyusun orientasi kamera dan translasi berdasarkan posisi kamera:

$$V = \begin{bmatrix} s_x & u_x & -f_x & 0 \\ s_y & u_y & -f_y & 0 \\ s_z & u_z & -f_z & 0 \\ -\mathbf{s} \cdot \mathbf{eye} & -\mathbf{u} \cdot \mathbf{eye} & \mathbf{f} \cdot \mathbf{eye} & 1 \end{bmatrix} \quad (13)$$

3.5 Projection Matrix

Projection matrix menggunakan perspektif. Dengan field of view vertikal $fovy$, aspect ratio a , near plane n , dan far plane f , nilai dasar perspektif adalah:

$$q = \frac{1}{\tan\left(\frac{fovy}{2}\right)} \quad (14)$$

Matriks projection yang digunakan program adalah:

$$P = \begin{bmatrix} \frac{q}{a} & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & -1 \\ 0 & 0 & \frac{2fn}{n-f} & 0 \end{bmatrix} \quad (15)$$

Urutan transformasi akhir pada vertex shader adalah:

$$\mathbf{p}_{clip} = P \cdot V \cdot M \cdot \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} \quad (16)$$

3.6 Normal Matrix

Normal tidak cukup hanya dikalikan dengan model matrix biasa, terutama jika ada scaling. Oleh karena itu vertex shader menggunakan normal matrix:

$$N = (M_{3 \times 3}^{-1})^T \quad (17)$$

Normal dunia dihitung dengan:

$$\mathbf{n}_{world} = \frac{N \cdot \mathbf{n}_{model}}{\|N \cdot \mathbf{n}_{model}\|} \quad (18)$$

3.7 Phong Lighting

Pencahayaan Phong pada fragment shader terdiri dari ambient, diffuse, dan specular. Arah cahaya dan arah pandang dihitung sebagai:

$$\mathbf{l} = \frac{\mathbf{p}_{light} - \mathbf{p}_{world}}{\|\mathbf{p}_{light} - \mathbf{p}_{world}\|} \quad (19)$$

$$\mathbf{v} = \frac{\mathbf{p}_{camera} - \mathbf{p}_{world}}{\|\mathbf{p}_{camera} - \mathbf{p}_{world}\|} \quad (20)$$

Komponen diffuse:

$$I_d = \max(\mathbf{n} \cdot \mathbf{l}, 0) \quad (21)$$

Arah refleksi cahaya:

$$\mathbf{r} = 2(\mathbf{n} \cdot \mathbf{l})\mathbf{n} - \mathbf{l} \quad (22)$$

Komponen specular:

$$I_s = \begin{cases} \max(\mathbf{v} \cdot \mathbf{r}, 0)^\alpha, & I_d > 0 \\ 0, & I_d = 0 \end{cases} \quad (23)$$

dengan α sebagai nilai **shininess**. Warna akhir fragment dihitung dengan:

$$\mathbf{C}_{final} = \mathbf{L}_{ambient}\mathbf{M}_{ambient} + I_d\mathbf{L}_{diffuse}\mathbf{M}_{diffuse} + I_s\mathbf{L}_{specular}\mathbf{M}_{specular} + \mathbf{C}_{rim} \quad (24)$$

3.8 Variasi Warna Berdasarkan Tinggi

Eksperimen fragment shader menambahkan variasi warna berdasarkan tinggi vertex. Rumus yang dipakai adalah:

$$b = 0.5 + 0.5 \sin(18y) \quad (25)$$

Nilai b dipakai untuk mencampur intensitas warna diffuse:

$$\mathbf{M}'_{diffuse} = \mathbf{M}_{diffuse} \cdot (0.78(1 - b) + 1.18b) \quad (26)$$

4 Implementasi

4.1 Struktur File

File utama proyek adalah:

- `src/main.cpp`: berisi seluruh kode program, mulai dari struktur data, loader OBJ, shader, setup OpenGL, setup VAO/VBO, input, dan render loop.
- `TungTungTungSahur.obj`: file model 3D yang dimuat ke scene.
- `Makefile`: konfigurasi kompilasi dan perintah menjalankan program.
- `README.md`: dokumentasi singkat build, run, dan kontrol.
- `report.tex`: file laporan LaTeX ini.

4.2 Struktur Data Dasar

Program mendefinisikan beberapa struktur data sederhana di `src/main.cpp`. Struktur `Vec3` digunakan untuk menyimpan vektor 3D, `Mat4` digunakan untuk matriks 4x4, `Vertex` menyimpan posisi dan normal, sedangkan `Mesh` menyimpan daftar vertex hasil pemuatan OBJ, titik tengah model, dan skala model.

```
struct Vertex {
    Vec3 position;
    Vec3 normal;
};

struct Mesh {
    std::vector<Vertex> vertices;
    Vec3 center;
    float scale = 1.0f;
};
```

Struktur `Material` dipakai untuk menyimpan properti material Phong: ambient, diffuse, specular, dan shininess.

```
struct Material {
    Vec3 ambient;
    Vec3 diffuse;
    Vec3 specular;
```

```
float shininess = 32.0f;
};
```

4.3 Implementasi Vertex dan Fragment Shader

Shader ditulis langsung sebagai string GLSL di dalam `main.cpp`. Shader untuk mesh terdiri dari `kMeshVertexShader` dan `kMeshFragmentShader`. Program mengompilasi shader dengan fungsi `compileShader()` dan menautkannya menjadi program shader dengan `makeProgram()`.

Vertex shader menerima dua atribut utama dari VAO, yaitu posisi vertex dan normal vertex.

```
layout (location = 0) in vec3 aPosition;
layout (location = 1) in vec3 aNormal;
```

Vertex shader mengubah posisi dari koordinat lokal model menjadi koordinat dunia menggunakan `uModel`, lalu mengubahnya ke clip space memakai `uView` dan `uProjection`.

```
vec4 world = uModel * vec4(aPosition, 1.0);
vWorldPos = world.xyz;
gl_Position = uProjection * uView * world;
```

Normal vertex dihitung memakai normal matrix:

```
vNormal = transpose(inverse(mat3(uModel))) * aNormal;
```

Normal matrix digunakan agar arah normal tetap benar ketika model mengalami transformasi seperti scaling. Hasil dari vertex shader, yaitu posisi dunia, normal, dan tinggi vertex, dikirim ke fragment shader.

4.4 Implementasi Phong Model

Fragment shader menerapkan model pencahayaan Phong. Di dalam shader terdapat dua struktur GLSL, yaitu `Material` dan `Light`. Struktur `Material` berisi karakter permukaan objek, sedangkan struktur `Light` berisi posisi dan warna cahaya.

```
struct Material {
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};

struct Light {
    vec3 position;
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
};
```

Komponen diffuse dihitung dari dot product antara normal permukaan dan arah cahaya. Jika permukaan menghadap cahaya, nilai diffuse akan semakin besar.

```
float diff = max(dot(n, lightDir), 0.0);
```

Komponen specular dihitung dari arah pandang kamera dan arah pantulan cahaya. Nilai shininess menentukan seberapa tajam highlight pada permukaan.

```
vec3 reflectDir = reflect(-lightDir, n);  
float spec = diff > 0.0  
    ? pow(max(dot(viewDir, reflectDir), 0.0), uMaterial.shininess)  
    : 0.0;
```

Warna akhir objek merupakan gabungan ambient, diffuse, specular, dan sedikit rim light statis agar bentuk model lebih mudah terlihat.

```
vec3 color = ambient + diffuse + specular + rimLight;  
fragColor = vec4(color, 1.0);
```

4.5 Eksperimen Shader dan Material

Untuk memenuhi bagian eksperimen vertex dan fragment shader, fragment shader diberi variasi warna berdasarkan tinggi vertex. Variasi ini tidak bergerak, sehingga scene tetap stabil.

```
float bands = 0.5 + 0.5 * sin(vHeight * 18.0);  
vec3 patternedDiffuse = uMaterial.diffuse * mix(0.78, 1.18, bands);
```

Program juga menyediakan empat preset material pada `kMaterials`. Material dapat diganti saat program berjalan dengan tombol 1 sampai 4. Setiap material memiliki nilai ambient, diffuse, specular, dan shininess yang berbeda, sehingga tampilan objek dapat berubah dari material kayu, emas, cyan, hingga abu-abu metalik.

```
static const std::vector<Material> kMaterials = {  
    {{0.18f, 0.08f, 0.03f}, {0.72f, 0.36f, 0.13f}, {0.35f, 0.22f, 0.12f}, 24.0f  
    },  
    {{0.10f, 0.08f, 0.05f}, {0.78f, 0.57f, 0.24f}, {0.95f, 0.82f, 0.45f}, 72.0f  
    },  
    {{0.02f, 0.08f, 0.09f}, {0.05f, 0.55f, 0.62f}, {0.55f, 0.90f, 0.95f}, 96.0f  
    },  
    {{0.04f, 0.04f, 0.045f}, {0.28f, 0.30f, 0.32f}, {0.85f, 0.88f, 0.90f},  
    128.0f},  
};
```

Pengiriman material ke shader dilakukan melalui fungsi `setMaterialUniforms()`.

```
static void setMaterialUniforms(GLuint program, const Material& material) {  
    setVec3Uniform(program, "uMaterial.ambient", material.ambient);  
    setVec3Uniform(program, "uMaterial.diffuse", material.diffuse);  
    setVec3Uniform(program, "uMaterial.specular", material.specular);  
    glUniform1f(glGetUniformLocation(program, "uMaterial.shininess"),  
        material.shininess);  
}
```

4.6 Fungsi Load Model 3D

Model 3D dimuat menggunakan fungsi `loadObj()`. Fungsi ini membuka file `TungTungTungSahur.obj`, membaca file baris demi baris, lalu memproses tag OBJ seperti `v`, `vn`, dan `f`.

- `v`: posisi vertex.
- `vn`: normal vertex.
- `f`: face atau sisi model.

Index vertex dan normal diproses oleh fungsi `parseObjIndex()` dan `parseNormalIndex()`. Face dengan lebih dari tiga vertex dipecah menjadi beberapa segitiga menggunakan metode triangle fan.

```
for (std::size_t i = 1; i + 1 < face.size(); ++i) {
    std::string tri[3] = {face[0], face[i], face[i + 1]};
    Vertex out[3];
    ...
}
```

Jika model tidak menyediakan normal untuk suatu face, program menghitung normal dengan cross product.

```
Vec3 faceNormal = normalize(cross(
    out[1].position - out[0].position,
    out[2].position - out[0].position
));
```

Selain membuat daftar vertex, loader juga menghitung bounding box model. Bounding box digunakan untuk mencari titik tengah dan skala model agar objek muncul di tengah scene dengan ukuran yang sesuai.

```
mesh.center = (minP + maxP) * 0.5f;
Vec3 extent = maxP - minP;
float maxExtent = std::max(extent.x, std::max(extent.y, extent.z));
mesh.scale = maxExtent > 0.0f ? 2.8f / maxExtent : 1.0f;
```

4.7 Penggunaan VAO dan VBO

Setelah model dimuat, vertex dari `mesh.vertices` dikirim ke GPU. Program membuat VAO untuk menyimpan konfigurasi atribut vertex dan VBO untuk menyimpan data vertex di GPU.

```
glGenVertexArrays(1, &meshVao);
glGenBuffers(1, &meshVbo);
glBindVertexArray(meshVao);
glBindBuffer(GL_ARRAY_BUFFER, meshVbo);
glBufferData(GL_ARRAY_BUFFER,
    static_cast<GLsizeiptr>(mesh.vertices.size() * sizeof(Vertex)),
    mesh.vertices.data(),
    GL_STATIC_DRAW);
```


Atribut posisi dikaitkan dengan location 0, sedangkan normal dikaitkan dengan location 1. Keduanya sesuai dengan deklarasi input pada vertex shader.

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), reinterpret_cast<void*>(offsetof(Vertex, position)));
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), reinterpret_cast<void*>(offsetof(Vertex, normal)));
glEnableVertexAttribArray(1);
```

Pada render loop, mesh digambar dengan `glDrawArrays()`.

```
glBindVertexArray(meshVao);
glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(mesh.vertices.size()));
```

4.8 Setup Model, View, dan Projection Matrix

Program memisahkan transformasi menjadi tiga fungsi. Pemisahan ini membuat pipeline grafika lebih jelas: model matrix mengatur objek, view matrix mengatur kamera, dan projection matrix mengatur perspektif.

4.8.1 Model Matrix

Model matrix dibuat oleh `makeModelMatrix()`. Fungsi ini memindahkan model ke pusat scene dan menerapkan skala dari hasil perhitungan bounding box.

```
static Mat4 makeModelMatrix(const Mesh& mesh) {
    return multiply(scale(mesh.scale), translate(mesh.center * -1.0f));
}
```

4.8.2 View Matrix

View matrix dibuat dari posisi kamera dan target. Fungsi `makeViewMatrix()` memanggil `lookAt()` dengan arah atas (0, 1, 0).

```
static Mat4 makeViewMatrix(Vec3 camera, Vec3 target) {
    return lookAt(camera, target, {0.0f, 1.0f, 0.0f});
}
```

Kamera dapat dikontrol dengan keyboard. Tombol A/D memutar kamera, W/S mengatur kemiringan kamera, dan Q/E mengatur jarak kamera dari objek.

4.8.3 Projection Matrix

Projection matrix dibuat oleh `makeProjectionMatrix()`. Fungsi ini menggunakan perspective projection dengan field of view 45 derajat. Aspect ratio dihitung dari ukuran framebuffer agar tampilan tidak terdistorsi ketika ukuran window berubah.

```
static Mat4 makeProjectionMatrix(int width, int height) {
    float aspect = width > 0 && height > 0
        ? static_cast<float>(width) / static_cast<float>(height)
        : 1.0f;
```

```
    return perspective(45.0f * 3.14159265f / 180.0f,
        aspect, 0.05f, 100.0f);
}
```

Ketiga matriks tersebut dikirim ke shader sebelum objek digambar.

```
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uModel"),
    1, GL_FALSE, model.m);
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uView"),
    1, GL_FALSE, view.m);
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uProjection"),
    1, GL_FALSE, projection.m);
```

4.9 Render Loop dan Input

Render loop berjalan selama window belum ditutup. Setiap frame, program membaca input, mengambil ukuran framebuffer, menghitung posisi kamera, membuat matriks model/view/projection, membersihkan layar, lalu menggambar ground grid dan model. Ground grid dibuat statis, tanpa animasi. Objek juga dibuat stabil, tanpa efek beat atau scaling berkala.

Material dipilih melalui tombol 1 sampai 4.

```
for (int i = 0; i < static_cast<int>(kMaterials.size()); ++i) {
    if (glfwGetKey(window, GLFW_KEY_1 + i) == GLFW_PRESS) {
        materialIndex = i;
    }
}
```

5 Screenshot

Bagian ini disiapkan untuk screenshot dari program yang berjalan dalam beberapa konfigurasi. Screenshot akan diisi dari hasil program yang dijalankan secara lokal. Jalankan program dengan:

```
make run
```

Lalu tekan tombol 1, 2, 3, dan 4 untuk mengambil screenshot material yang berbeda. Simpan file screenshot ke folder **screenshots/** dengan nama berikut.

Placeholder screenshot: `screenshots/material-1.png`
Bagian ini diisi menggunakan screenshot dari program yang dijalankan.

Figure 1: Program berjalan dengan material preset 1.

Placeholder screenshot: `screenshots/material-2.png`
Bagian ini diisi menggunakan screenshot dari program yang dijalankan.

Figure 2: Program berjalan dengan material preset 2.

Placeholder screenshot: `screenshots/material-3.png`
Bagian ini diisi menggunakan screenshot dari program yang dijalankan.

Figure 3: Program berjalan dengan material preset 3.

Placeholder screenshot: `screenshots/material-4.png`
Bagian ini diisi menggunakan screenshot dari program yang dijalankan.

Figure 4: Program berjalan dengan material preset 4.

6 Kesimpulan

Program telah memenuhi tiga kebutuhan utama pada tugas. Vertex shader dan fragment shader sudah dibuat dengan model pencahayaan Phong. Objek 3D dimuat dari file OBJ dan dikirim ke GPU menggunakan VAO serta VBO. Transformasi scene juga sudah dipisahkan menjadi model matrix, view matrix, dan projection matrix. Selain itu, program menyediakan beberapa material yang dapat diganti saat runtime, sehingga konfigurasi tampilan objek dapat dibandingkan melalui screenshot.